

Chapter → 10

Object Oriented Programming

Solving a problem by creating object is one of the most popular approaches in programming. This is object-oriented programming.

● Class

A class is a blueprint for creating object combines info to create

a valid application ← Blank Form ⇒ Filled by a Student ⇒ application of the student

contains ↙ class ⇒ object instantiation ⇒ object
info to create a valid object

Syntax:

class Employee: # class name is written in pascal class
 # Method and variables

● Object

An object is an instantiation of a class. When class is defined a template (info) is defined. Memory is allocated only after object instantiation.

Objects of a given class can invoke the methods available to it without revealing the implementation detailed to the user -
Abstraction & Encapsulation.

● Modelling A Problem in OOPs

We identify the following in our problem :

- > Noun → class → Employee
- > Adjective → Attributes → name, age, salary
- > Verbs → Methods → getSalary(), increment()

● Class Attributes

An Attribute that belongs to the class rather than a particular object.

Example:

class Employee:

 company = "Google"

Specific to each class

harry = Employee()

Object Instantiation

harry.company

Employee.company = "Youtube"

Changing class Attribute

● Instance Attributes

An Attribute that belongs to the Instance (Object).

Assuming the class from the previous example:

harry.name = "Harry"

harry.salary = "30k"

Adding instance attributes

Note: Instance attributes take preference over class attributes during assignment & retrieval.

● Self Parameter

Self refers to the instance of the class. It is automatically passed with a function call from an object.

harvey.getSalary() # here self is harvey
equivalent to the Employee.getSalary(harvey)

The function getSalary() is defined as
class Employee:

```
    company = "Google"  
    def getSalary(self):  
        print("Salary is not there")
```

● Static Method

Sometimes we need a function that does not use the self-parameter. We can define a static method like this:

@staticmethod

decorator to mark greet as a static method

```
def greet():  
    print("Hello user")
```

● __init__() Constructor

__init__() is a special method which is first run as soon as the object is created.

__init__() method is also known as constructor

It takes self argument and can also take further arguments.

Example:

```
class Employee:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def getSalary(self):
```

```
        ...
```

```
harry = Employee("Harry")
```

Chapter - 10

Practice - Set

- 1) Create a class "Programmer" for storing information of few Programmers working at Microsoft.
- 2) Write a class "Calculator" Capable of finding square, cube, and square root of a number.
- 3) Create a class with class attribute `a`; Create an object from it and set '`a`' directly using `object.a = 0`. Does this change the class attributes.
- 4) Add a static method in Problem 2, to greet the user with 'Hello'.
- 5) Write a class Train which has methods to book a ticket, get status (no. of seats) and get fare information of train running under Indian railways.
- 6) Can you change the self Parameter inside a class to something else (say "Harry"). Try changing self to "self" or "Harry" and see the effects.